

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Оглавление

Условные операторы	2
Оператор if	2
Моржовый оператор (walrus operator) :=	4
Оператор ветвления else	4
Оператор ветвления elif	5
Вложенные операторы	5
Тернарный оператор	6
Общие рекомендации по написанию кода (II)	7

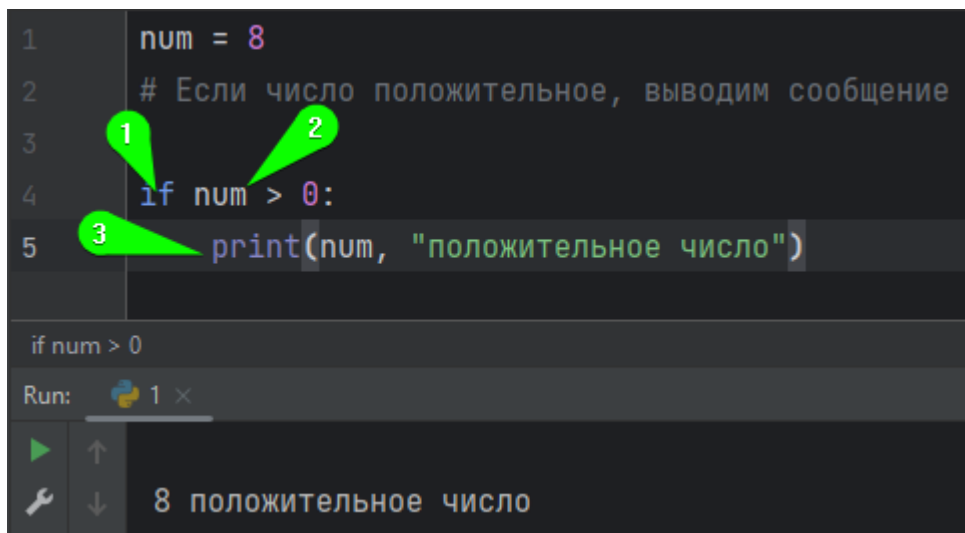
Условные операторы

На прошлых уроках мы познакомились с основными типами данных, а теперь пришло время узнать, с помощью каких операторов мы можем осуществлять управление нашим кодом.

При написании программ мы часто можем столкнуться с неопределённостью: ввёл пользователь число или строку, заполнен список данными или нет, есть доступ в интернет или отсутствует? Для каждого такого варианта событий нам нужно описать свой код, попытавшись «закрыть» все возможные случаи.

Оператор if

Условный оператор if (если) в python как раз и позволяет нам изменять поведение нашего кода в зависимости от текущего состояния. Условие if состоит из нескольких частей: самого **условного оператора if (1)**, **условия (2)** и **тела условия (3)**.



```
1 num = 8
2 # Если число положительное, выводим сообщение
3 if num > 0:
4     print(num, "положительное число")
```

The screenshot shows a code editor with the following code: `num = 8`, a comment `# Если число положительное, выводим сообщение`, and an `if` statement `if num > 0: print(num, "положительное число")`. Three green callouts with numbers 1, 2, and 3 point to the `if` keyword, the condition `num > 0`, and the `print` statement respectively. Below the code, a console window shows the output: `8 положительное число`.

После оператора if мы должны добавить пробел и указать условие. Тело помещается вложенным в заголовок и может состоять из одной или нескольких строк. Чтобы программа поняла вложенность и правильно выполнялась, необходимо сделать отступ в виде **4 пробелов** для каждой строки кода.

Результат выражения (условия), расположенного после условного оператора if будет приведён к булевому типу, то есть мы получим либо **True**, либо **False**. Напомню, что мы получим False в следующих ситуациях: передав пустую строку/список/словарь/множество, передав тип данных None, 0 и собственном сам False.

Вы можете представить себе вызов функции bool, с аргументом – условием

```
if bool(num > 0):
```

Если выражение после if будет истинно, то есть мы получим в результате операций True, мы провалимся в тело условия – в примере выше мы распечатали строку с переменной num и строку «положительное число».

Все логические операторы описанные в уроке Переменные актуальны для условия if. Допускается объединение нескольких условий через оператор and

```
1 if 8 > 0 and 'condition':
2     print(True)
```

if 8 > 0 and 'condition'

Run: 1 x

True

В этом коде мы объединили два условия: первое условие «восемь больше нуля» вернуло нам истину – True, второе так же вернуло истину, потому как строка была не пустой, как мы помним пустая строка вернула бы нам False.

Обратите внимание на использование оператора or – при истинности левого условия, правое проверяться не будет.

```
1 if 8 > 0 or 1 / 0:
2     print('Левое условие истинно')
3     print('Правое условие не выполнялось')
```

if 8 > 0 or 1 / 0

Run: 1 x

Левое условие истинно

Правое условие не выполнялось

В любом другом случае деление на ноль вернуло бы нам ошибку

```
1 if 8 > 0 and 1 / 0:
2     print('Левое условие истинно')
3     print('Правое условие не выполнялось')
```

if 8 > 0 and 1 / 0

Run: 1 x

Traceback (most recent call last):

File "D:\Dropbox\Документы\Coding\Python\lessons\1.py", line 1, in <module>

if 8 > 0 and 1 / 0:

~~~~~

ZeroDivisionError: division by zero

## Моржовый оператор (walrus operator) :=

Начиная с версии 3.8 в python появился моржовый оператор := названный так из-за схожести с клыками и глазами моржа.

Используется он для того, чтобы сразу в условии if создать переменную, присвоив ей результат следующей за ним операции. Таким образом, вместо двух строк кода - одной для объявления переменной и второй для проверки, мы можем создать только одну:

The image shows two side-by-side screenshots of a Python IDE. The left screenshot shows a traditional if-statement: `num = int(input("Введите число: "))` followed by `if num == 0:` and `print("Был введен ноль")`. The right screenshot shows the same logic using the walrus operator: `if (num := int(input("Введите число: "))):` followed by `print("Был введен ноль")`. Both screenshots show the program running and outputting "Введите число: 0" and "Был введен ноль".

Обратите внимание, что **присваивание должно быть убрано в круглые скобки**.

Каждый раз, когда у вас в коде встречается объявление переменной и далее её проверка, объединяйте эти строки в условии if.

## Оператор ветвления else

Для того, чтобы обработать ситуацию, когда условие оказалось ложным, используется блок else – в него мы попадаем если условие в if оказалось ложным (False). Для разделения блоков также используются отступы.

The image shows a screenshot of a Python IDE with the following code: `num = -1`, `# Если число положительное, выводим сообщение из тела if`, `if num > 0:`,  `print(num, "положительное число")`, `# Если число отрицательное, выводим сообщение из тела else`, `else:`,  `print(num, "отрицательное число")`. The output shows "-1 отрицательное число".



## Оператор ветвления elif

Если одного условия в `if` недостаточно, можно добавить ниже ещё нужное количество, но следующие проверяются уже оператором `elif`: сокращённо `else + if`, то есть “иначе если”.

```

1  num = 0
2  # Если число положительное, выводим сообщение из тела if
3  if num > 0:
4      print(num, "положительное число")
5  # Иначе если число равно нулю, выводим сообщение из тела elif
6  elif num == 0:
7      print(num, "ноль")
8  # Если число отрицательное, выводим сообщение из тела else
9  else:
10     print(num, "отрицательное число")

```

else

Run: 1 x

0 ноль

Мы можем добавить необходимое количество таких условий.

## Вложенные операторы

**if...elif...else** можно размещать внутри других операторов **if...elif...else**. Любое количество этих операторов может быть вложено в друг друга. Степень вложенности определяется отступами. То есть если мы хотим вложить один оператор в другой, то нужно перед заголовком сделать отступ в 4 пробела.

Предыдущий пример перепишем, и сделаем со вложенными операторами.

```

# В зависимости от введённого числа будет выводиться сообщение

num = int(input("Введите число: "))
if num >= 0:
    if num == 0:
        print(num, "ноль")
    else:
        print(num, "положительное число")
else:
    print(num, "отрицательное число")

```

#  
# Это вложенный код  
#  
#

## Тернарный оператор

Во многих языках программирования, в том числе и в python, существуют конструкции, позволяющие убрать многострочные решения в одну строку кода. Одним из таких способов является тернарный оператор - он даёт возможность описать условие `if` и блок ветвления `else` в одной строке. То есть мы можем заменить условие:

```
if выражение:
    результат если True
else:
    результат если False
```

на следующую запись:

```
результат если True if выражение else результат если False
```

Подобным способом компоновки кода имеет смысл пользоваться, когда выражение поместится в одной строке. Если строк будет больше, ценность такого преобразования значительно снижается.

## Общие рекомендации по написанию кода (II)

Создавая длинные условия, старайтесь каждый раз на первое место ставить негативный вариант. Таким образом вы в начале, когда сразу будете видеть условие, которого нужно избежать. Сравните 2 варианта одного и того же кода:

```
vuln_scanners = ("Nessus", "XSpider", "Retina", "MaxPatrol")
net_scanners = ("nmap", "masscan", "IPVOID", "Advanced Port Scanner")
owasp_top_3 = ("Broken Access Control", "Cryptographic Failures", "Injection")

user_answer = input("User input:")

if user_answer in vuln_scanners:
    print("Using scanner", user_answer)
elif user_answer in net_scanners:
    print("Using network scanner", user_answer)
elif user_answer in owasp_top_3:
    print("Checking vulnerability", user_answer)
else:
    print("User_input error!")
```

```
vuln_scanners = ("Nessus", "XSpider", "Retina", "MaxPatrol")
net_scanners = ("nmap", "masscan", "IPVOID", "Advanced Port Scanner")
owasp_top_3 = ("Broken Access Control", "Cryptographic Failures", "Injection")

user_answer = input("User input:")

if user_answer not in vuln_scanners + net_scanners + owasp_top_3:
    print("User_input error!")
else:
    if user_answer in vuln_scanners:
        print("Using scanner", user_answer)
    elif user_answer in net_scanners:
        print("Using network scanner", user_answer)
    else:
        print("Checking vulnerability", user_answer)
```

В первом случае мы видим действие, которое произойдёт при ошибочном вводе, только в самом низу условия, и нам придётся пробежаться глазами по всем вариантам прежде, чем мы узнаем, что произойдёт в негативном случае.

Второй же вариант сразу описывает ситуацию, когда пользовательский ввод не присутствует в нужных нам кортежах, а вложенность ещё больше помогает сориентироваться в коде.